Name: Mahitha Kalaga

Email ID: Kalagam1@udayton.edu

A SECURE AND ROBUST CHAT SYSTEM IN GOLANG AND NODE.JS

INTRODUCTION

URL: https://bitbucket.org/mahithasad/secad-mahi/commits/741cbc8480dbc1fe4f660cee32406f416bd249bc

The chat application used in this project consists of a server “ChatServer.go” and a client “chatclient.js”, which are designed to enable real-time conversation among multiple users across a TCP connection. The server, which is in Go, provides authentication for the user, maintains lists of logged-in clients, and broadcasts both public and private messages with support for only authenticated users to participate in the chat. The client, built on Node.js, provides an interactive interface for logging in, sending and receiving messages, and receiving updates, including functionalities such as requests for a list of users and logout, as evident from various test cases.

DESIGN:

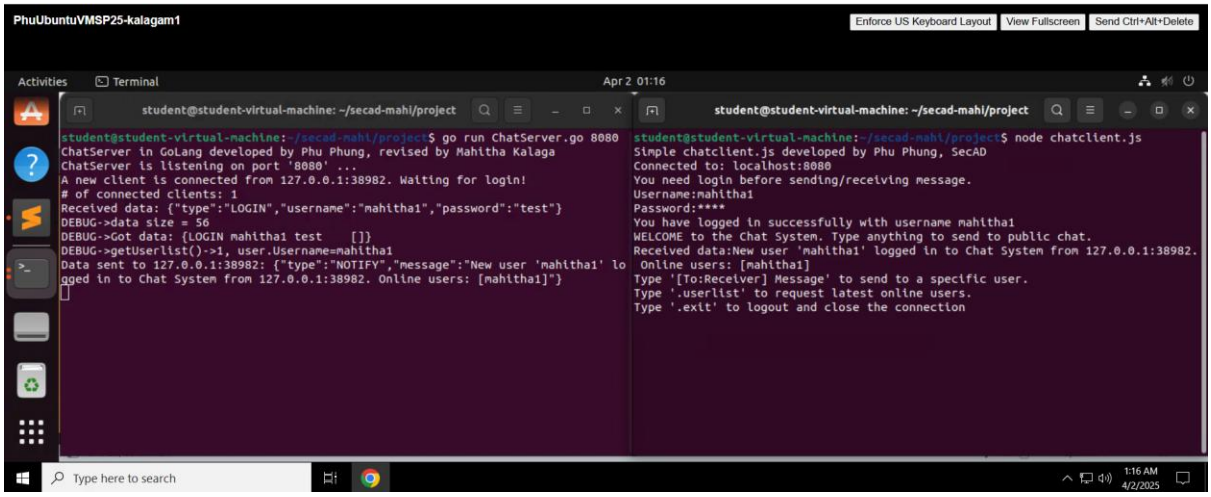
```
mahitha1 (Client)    mahitha2 (Client)    mahitha2 (Client)    Server (ChatServer.go)
    |                    |                    |                    |
    |---- Connect ------>|                    |                    |
    |                    |                    |                    | (Accept connection)
    |                    |                    |                    | (Add mahitha1 to authClients)
    |---- LOGIN -------->|                    |                    |
    |  {type: "LOGIN",   |                    |                    |
    |   username: "mahitha1", |               |                    |
    |   password: "test"}|                    |                    |
    |                    |                    |                    | (Verify credentials)
    |                    |                    |                    | (Set mahitha1.loggedIn = true)
    |<--- AUTH_SUCCESS ---|                    |                    |
    |  {type: "AUTH_SUCCESS", |               |                    |
    |   message: "Welcome..."}|               |                    |
    |                    |                    |                    |
    |                    |---- Connect ------>|                    |
    |                    |                    |                    | (Accept connection)
    |                    |                    |                    | (Add mahitha2 to authClients)
    |                    |---- LOGIN -------->|                    |
    |                    |  {type: "LOGIN",   |                    |
    |                    |   username: "mahitha2", |               |
    |                    |   password: "test"}|                    |
    |                    |                    |                    | (Verify credentials)
    |                    |                    |                    | (Set mahitha2.loggedIn = true)
    |                    |<--- AUTH_SUCCESS ---|                    |
    |                    |  {type: "AUTH_SUCCESS", |               |
    |                    |   message: "Welcome..."}|               |
    |                    |                    |                    |
    |                    |                    |                    | (Broadcast NOTIFY to all logged-in)
    |<--- NOTIFY ---------|                    |                    |
    |  {type: "NOTIFY",  |                    |                    |
    |   message: "New user 'mahitha2' logged in to Chat System from 127.0.0.1:147014. Online users: [mahitha1, mahitha2]"}|
    |                    |                    |                    |
    |                    |                    |---- Connect ------>|
    |                    |                    |                    | (Accept connection)
    |                    |                    |                    | (Add user to authClients)
    |                    |                    |---- LOGIN -------->|
    |                    |                    |  {type: "LOGIN",   |
    |                    |                    |   username: "mahitha2", |
    |                    |                    |   password: "hskse"}|
    |                    |                    |                    | (Verify credentials - Failed)
    |                    |                    |<--- AUTH_FAILED ---|
    |                    |                    |  {type: "AUTH_FAILED", |
    |                    |                    |   message: "Invalid username or password"}|
```

```
|                    |                    |   username: "mahitha2", |
|                    |                    |   password: "hskse"}|
|                    |                    |                | (Verify credentials - Failed)
|                    |                    |<--- AUTH_FAILED ---|
|                    |                    |  {type: "AUTH_FAILED", |
|                    |                    |   message: "Invalid username or password"}|
|                    |                    |                |
|---- PUBLIC -------->|                    |                |
|  {type: "PUBLIC",   |                    |                |
|   message: "Hi all"}|                    |                |
|                    |                    |                | (Broadcast PUBLIC to all logged-in except sender)
|                    |<--- PUBLIC ---------|                |
|                    |  {type: "PUBLIC",   |                |
|                    |   sender: "mahitha1", |              |
|                    |   message: "Hi all"}|                |
|                    |                    |                |
|---- PRIVATE ------->|                    |                |
|  {type: "PRIVATE",  |                    |                |
|   target: "mahitha2", |                  |                |
|   message: "Hi saying privately"}|       |                |
|                    |                    |                | (Send PRIVATE to mahitha2)
|                    |<--- PRIVATE --------|                |
|                    |  {type: "PRIVATE",  |                |
|                    |   sender: "mahitha1", |              |
|                    |   message: "Hi saying privately"}|       |
|                    |                    |                |
|                    |---- PUBLIC -------->|                |
|                    |  {type: "PUBLIC",   |                |
|                    |   message: "Hi all"}|                |
|                    |                    |                | (Broadcast PUBLIC to all logged-in except sender)
|<--- PUBLIC ---------|                    |                |
|  {type: "PUBLIC",   |                    |                |
|   sender: "mahitha2", |                  |                |
|   message: "Hi all"}|                    |                |
|                    |                    |                |
|---- LOGOUT -------->|                    |                |
|  {type: "LOGOUT"}   |                    |                |
|                    |                    |                | (Remove mahitha1 from authClients)
|                    |                    |                | (Broadcast NOTIFY to all logged-in)
|                    |<--- NOTIFY ---------|                |
|                    |  {type: "NOTIFY",   |                |
|                    |   message: "New user 'mahitha1' logged out to Chat System from 127.0.0.1:147014. Online users: [mahitha2]"}|
|                    |                    |                |
|<--- (Connection closed) ---|             |                |
```

TEST CASES AND DEMO

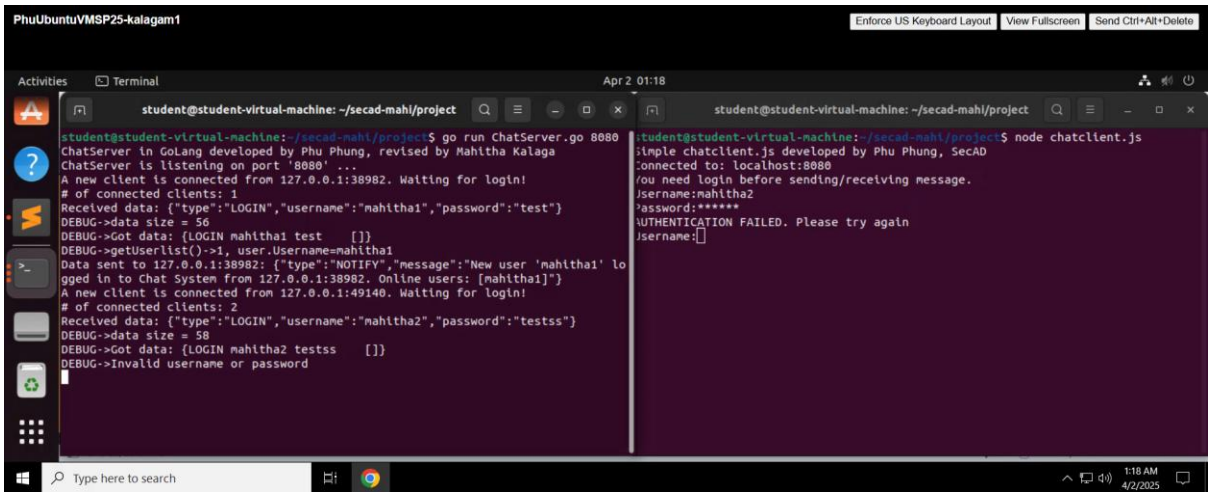
Test case 1: A user has been login successfully at the server side (left) and at the client side (right)

- A client connects to the server and sends a login request with valid credentials (username: mahitha1, password: test).
- The server authenticates the user, marks them as logged in, and broadcasts a notification to all clients about the new user.
- The client receives a success message (AUTH_SUCCESS) and can now participate in the chat system



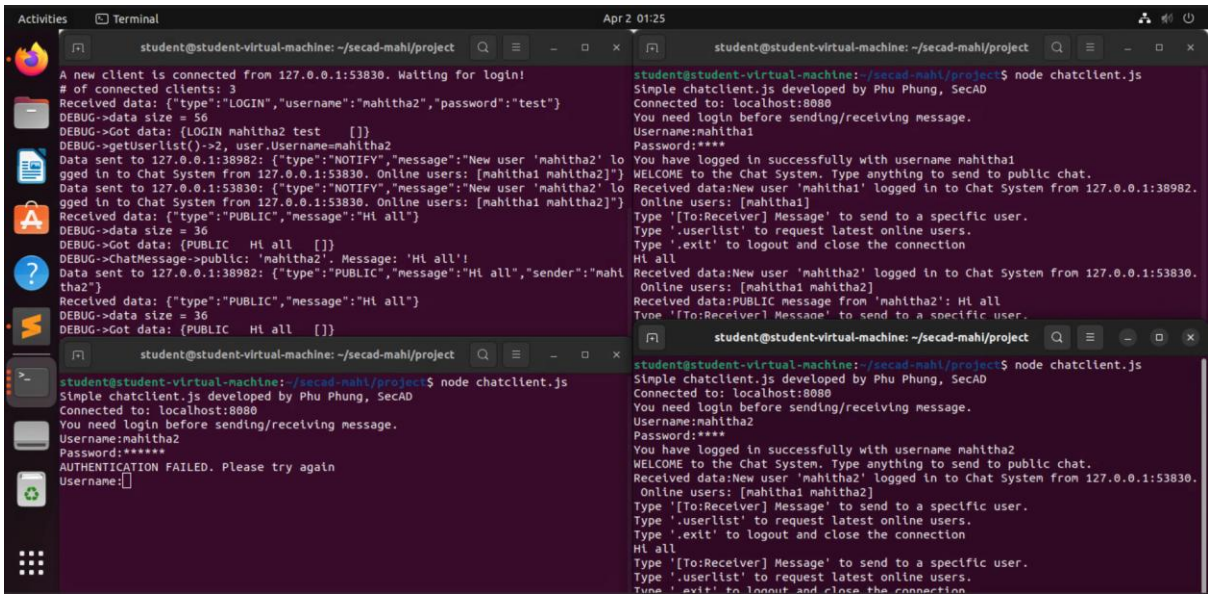
Test case 2: A user cannot log in due to an invalid username or password

- A client attempts to log in with incorrect credentials (username: mahitha2, password: 123456).
- The server rejects the login attempt, logging the failure, and sends an AUTH_FAILED message to the client.
- The client displays "AUTHENTICATION FAILED" and prompts the user to retry with valid credentials.



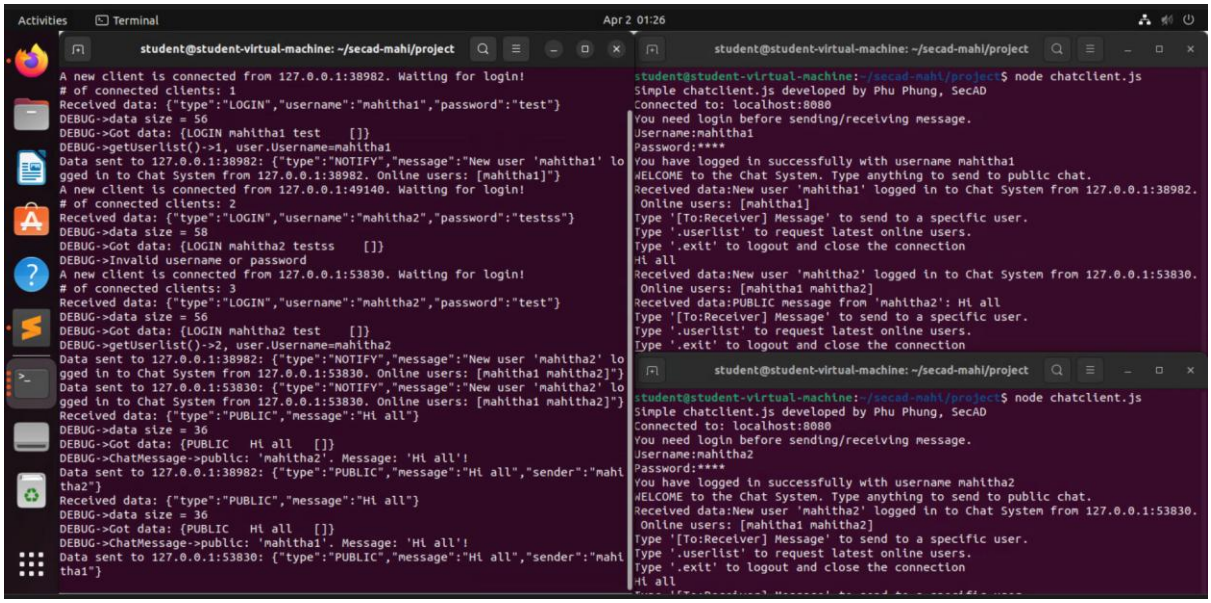
Test case 3: Logged-in users can receive public chat messages while unlogged-in users cannot

- Client 1 (mahitha1) logs in and sends a public message (Hi all) to all logged-in users.
- Client 2 (mahitha2), also logged in, receives the message, while an unlogged-in client (mahitha2 with wrong password) does not.
- The server ensures that only authenticated, logged-in clients receive public messages.



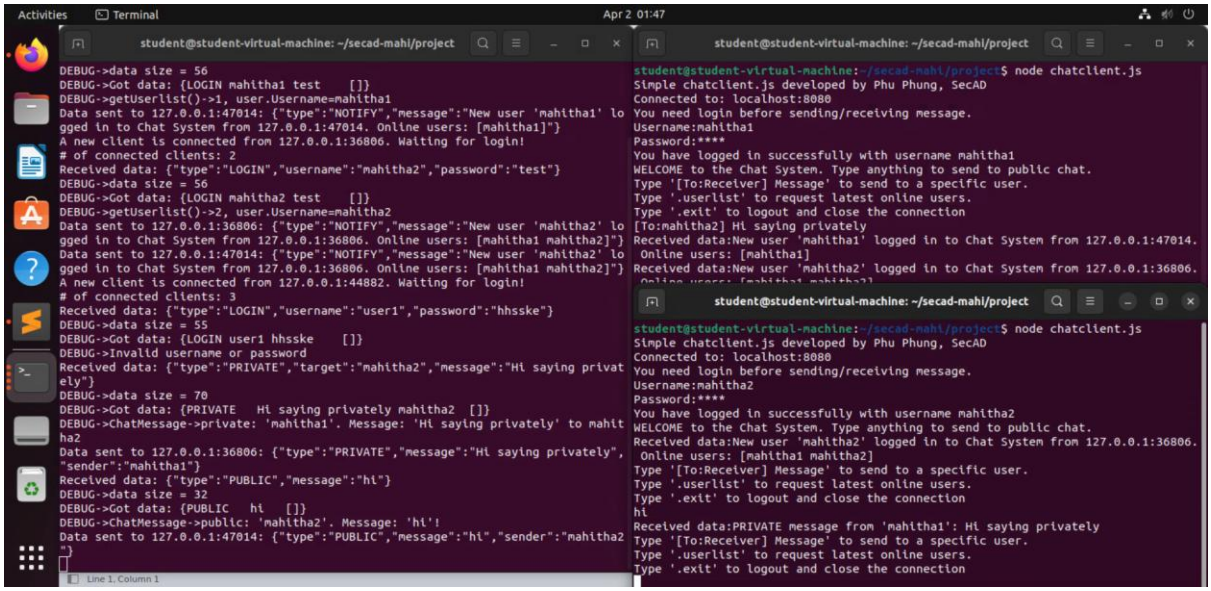
Test case 4 & 5: All logged-in users receive a notification with the updated user list when a new user is logged in successfully. Public chat messages are sent to all logged-in users

- Client 2 (mahitha2) logs in, triggering the server to notify all logged-in clients (mahitha1, mahitha2) of the updated user list.
- Client 2 sends a public message (Hi all), which is broadcast to mahitha1 and mahitha2, but not to unlogged-in clients.
- The server maintains the user list and ensures public messages reach all logged-in clients except the sender.



Test case 6: Private messages can only be received by the individual receiver

- Client 1 (mahitha1) sends a private message (Hi saying privately) to Client 2 (mahitha2).
- The server delivers the message only to mahitha2, ensuring Client 3 does not receive it.
- This demonstrates the server's ability to handle targeted private communication securely.



SOURCE CODE

ChatServer.go:

```
package main
import (
    "encoding/json"
    "fmt"
    "net"
    "os"
    "strings"
    "sync"
)
var accounts = map[string]string{
    "mahitha1": "test",
    "mahitha2": "test",
    "user": "test",
}
const BUFFERSIZE int = 1024
type Message struct {
    Type     string   json:"type"
    Username string   json:"username,omitempty"
    Password string   json:"password,omitempty"
    Message  string   json:"message,omitempty"
    Target   string   json:"target,omitempty"
    Sender   string   json:"sender,omitempty"
    Users    []string json:"users,omitempty"
}
type Client struct {
    conn     net.Conn
    username string
   loggedIn bool
```

```
}

var authClients = struct {
    sync.Mutex
    clients map[*Client]bool
}{clients: make(map[*Client]bool)}

var newclient = make(chan *Client)
var lostclient = make(chan *Client)

func main() {
    if len(os.Args) != 2 {
        fmt.Printf("Usage: %s <port>\n", os.Args[0])
        os.Exit(0)
    }
    port := os.Args[1]
    if len(port) > 5 {
        fmt.Println("Invalid port value. Try again!")
        os.Exit(1)
    }
    server, err := net.Listen("tcp", ":"+port)
    if err != nil {
        fmt.Printf("Cannot listen on port '%s'!\n", port)
        os.Exit(2)
    }
    fmt.Printf("ChatServer in GoLang developed by Phu Phung, revised by Mahitha Kalaga
\n")
   fmt.Printf("ChatServer is listening on port '%s' ...\n", port)

    go func() {
        for {
```

```
            conn, err := server.Accept()
            if err != nil {
                fmt.Println("Error accepting connection:", err)
                continue
            }
            client := &Client{conn: conn, loggedIn: false}
            fmt.Printf("A new client is connected from %s. Waiting for login!\n",
conn.RemoteAddr().String())
            newclient <- client
        }
    }()

    for {
        select {
        case client := <-newclient:
            go clientGoroutine(client)
        case client := <-lostclient:
            authClients.Lock()
            if _, exists := authClients.clients[client]; exists {
                delete(authClients.clients, client)
                if client.loggedIn {
                    broadcast(Message{Type: "NOTIFY", Message: fmt.Sprintf("New user '%s'
logged out to Chat System from %s. Online users: %v", client.username,
client.conn.RemoteAddr().String(), getUserList())}, true)
                }
            }
            authClients.Unlock()
            fmt.Printf("# of connected clients: %d\n", len(authClients.clients))
        }
    }
}
```

```
func clientGoroutine(client *Client) {
    defer func() {
        client.conn.Close()
        lostclient <- client
    }()

    authClients.Lock()
    authClients.clients[client] = true
    authClients.Unlock()
    fmt.Printf("# of connected clients: %d\n", len(authClients.clients))

    var buffer [BUFFERSIZE]byte
    for {
        byteReceived, readErr := client.conn.Read(buffer[0:])
        if readErr != nil {
            return
        }

        clientData := strings.TrimSpace(string(buffer[:byteReceived]))
        fmt.Printf("Received data: %s\n", clientData)
        fmt.Printf("DEBUG->data size = %d\n", len(clientData))

        var msg Message
        if err := json.Unmarshal([]byte(clientData), &msg); err != nil {
            fmt.Printf("DEBUG->strings.Compare: non-login data\n")
            sendToClient(client, Message{Type: "ERROR", Message: "non-login data. Please
send login data first!"})
            continue
        }
```

```
        fmt.Printf("DEBUG->Got data: %v\n", msg)


        if !client.loggedIn {
            if msg.Type != "LOGIN" {
                fmt.Printf("DEBUG->strings.Compare: non-login data\n")
                sendToClient(client, Message{Type: "ERROR", Message: "non-login data. Please
send login data first!"})
                continue
            }


            if valid, message := checkLogin(msg); valid {
                client.username = msg.Username
                client.loggedIn = true
                fmt.Printf("DEBUG->getUserlist()->%d, user.Username=%s\n",
len(getUserList()), client.username)
                sendToClient(client, Message{Type: "AUTH_SUCCESS", Message: "Welcome to
the chat system!"})
                broadcast(Message{Type: "NOTIFY", Message: fmt.Sprintf("New user '%s' logged
in to Chat System from %s. Online users: %v", client.username,
client.conn.RemoteAddr().String(), getUserList())}, true)
            } else {
                fmt.Printf("DEBUG->Invalid username or password\n")
                sendToClient(client, Message{Type: "AUTH_FAILED", Message: message})
            }
        } else {
            switch msg.Type {
            case "PUBLIC":
                fmt.Printf("DEBUG->ChatMessage->public: '%s'. Message: '%s'!\n",
client.username, msg.Message)
                broadcast(Message{Type: "PUBLIC", Sender: client.username, Message:
msg.Message}, false)
            case "PRIVATE":
```

```
            fmt.Printf("DEBUG->ChatMessage->private: '%s'. Message: '%s' to %s\n",
client.username, msg.Message, msg.Target)
            sendPrivate(client, client.username, msg.Target, msg.Message)
        case "LOGOUT":
            return
        case "USERLIST":
            sendToClient(client, Message{Type: "USERLIST", Users: getUserList()})
        default:
            sendToClient(client, Message{Type: "ERROR", Message: "Unknown command"})
        }
    }
  }
}

func checkLogin(msg Message) (bool, string) {
  if storedPass, exists := accounts[msg.Username]; exists && storedPass == msg.Password {
    return true, "Authentication successful"
  }
  return false, "Invalid username or password"
}

func sendToClient(client *Client, msg Message) {
  data, err := json.Marshal(msg)
  if err != nil {
    fmt.Println("Error marshaling JSON:", err)
    return
  }
  data = append(data, '\n')
  _, writeErr := client.conn.Write(data)
  if writeErr != nil {
    fmt.Printf("Error sending to %s: %v\n", client.conn.RemoteAddr().String(), writeErr)
```

```
    }
}

func broadcast(msg Message, includeSender bool) {
    data, err := json.Marshal(msg)
    if err != nil {
        fmt.Println("Error marshaling JSON:", err)
        return
    }
    data = append(data, '\n')

    authClients.Lock()
    defer authClients.Unlock()

    for client := range authClients.clients {
        if !client.loggedIn {
            continue
        }
        if !includeSender && msg.Sender == client.username {
            continue
        }
        fmt.Printf("Data sent to %s: %s", client.conn.RemoteAddr().String(), string(data))
        _, writeErr := client.conn.Write(data)
        if writeErr != nil {
            fmt.Printf("Error broadcasting to %s: %v\n", client.conn.RemoteAddr().String(),
writeErr)
            go func(c *Client) { lostclient <- c }(client)
        }
    }
}
```

```
func sendPrivate(senderClient *Client, senderUsername, targetUsername, message string) {
    authClients.Lock()
    defer authClients.Unlock()

    targetFound := false
    for client := range authClients.clients {
        if client.loggedIn && client.username == targetUsername {
            targetFound = true
            msg := Message{Type: "PRIVATE", Sender: senderUsername, Message: message}
            data, err := json.Marshal(msg)
            if err != nil {
                fmt.Println("Error marshaling JSON:", err)
                continue
            }
            data = append(data, '\n')
            fmt.Printf("Data sent to %s: %s", client.conn.RemoteAddr().String(), string(data))
            _, writeErr := client.conn.Write(data)
            if writeErr != nil {
                fmt.Printf("Error sending private to %s: %v\n", client.conn.RemoteAddr().String(),
writeErr)
            }
        }
    }
    if !targetFound {
        sendToClient(senderClient, Message{Type: "ERROR", Message: fmt.Sprintf("User %s
not found", targetUsername)})
    }
}

func getUserList() []string {
    authClients.Lock()
```

```
    defer authClients.Unlock()
    var usernames []string
    for client := range authClients.clients {
        if client.loggedIn {
            usernames = append(usernames, client.username)
        }
    }
    return usernames
}
```

Chatclient.go:

```
const net = require('net');
const readlineSync = require('readline-sync');
const client = new net.Socket();
const HOST = 'localhost';
const PORT = 8080;
let username = '';
let authenticated = false;
let buffer = '';
function sendMessage(message) {
    const data = JSON.stringify(message) + '\n';
    client.write(data);
}
function login() {
    const usernameInput = readlineSync.question('Username:');
   const passwordInput = readlineSync.question('Password:', { hideEchoBack: true, mask: '*'
});
    username = usernameInput;
    const message = { type: 'LOGIN', username: usernameInput, password: passwordInput };
    sendMessage(message);
}
```

```
function handleChat() {
    console.log("Type '[To:Receiver] Message' to send to a specific user.");
    console.log("Type '.userlist' to request latest online users.");
    console.log("Type '.exit' to logout and close the connection");
    const input = readlineSync.question(", { keepWhitespace: true }).trim();

    if (input.startsWith('[To:')) {
        const match = input.match(/\[To:([^\]]+)\]\s*(.+)/);
        if (match) {
            const target = match[1];
            const message = match[2];
            sendMessage({ type: 'PRIVATE', target, message });
        } else {
            console.log('Invalid private message format. Use [To:Receiver] Message');
        }
    } else if (input === '.userlist') {
        sendMessage({ type: 'USERLIST' });
    } else if (input === '.exit') {
        sendMessage({ type: 'LOGOUT' });
        client.end();
        return false;
    } else if (input) {
        sendMessage({ type: 'PUBLIC', message: input });
    }
    return true;
}

client.connect(PORT, HOST, () => {
    console.log(Simple chatclient.js developed by Phu Phung, SecAD);
    console.log(Connected to: ${HOST}:${PORT});
```

```
    console.log('You need login before sending/receiving message.');
    login();
});

client.on('data', (data) => {
    buffer += data.toString();
    const messages = buffer.split('\n');
    buffer = messages.pop();

    for (const msg of messages) {
        if (msg.trim() === '') continue;
        try {
            const parsed = JSON.parse(msg);
            if (parsed.type === 'AUTH_SUCCESS') {
                console.log(You have logged in successfully with username ${username});
                console.log('WELCOME to the Chat System. Type anything to send to public
chat.');
                authenticated = true;
            } else if (parsed.type === 'AUTH_FAILED') {
                console.log('AUTHENTICATION FAILED. Please try again');
                login();
            } else if (parsed.type === 'PUBLIC') {
                console.log(Received data:PUBLIC message from '${parsed.sender}':
${parsed.message});
            } else if (parsed.type === 'PRIVATE') {
                console.log(Received data:PRIVATE message from '${parsed.sender}':
${parsed.message});
            } else if (parsed.type === 'USERLIST') {
                console.log(Received data: Online users: [${parsed.users.join(', ')}]);
            } else if (parsed.type === 'NOTIFY') {
                console.log(Received data:${parsed.message});
```

```
            } else if (parsed.type === 'ERROR') {
                console.log(Received data:${parsed.message});
            }
        } catch (err) {
            console.log('Error parsing message:', msg, err);
        }
    }
});

client.on('close', () => {
    console.log('Connection closed');
});

client.on('error', (err) => {
    console.error('Connection error:', err.message);
});

function startChatLoop() {
    if (authenticated) {
        if (!handleChat()) {
            return;
        }
    }
    setImmediate(startChatLoop);
}

client.on('connect', () => {
    startChatLoop();
});
```